

Experiment 3: Creation of a Simple Network Topology Using Open-Source Network Virtualization Tools

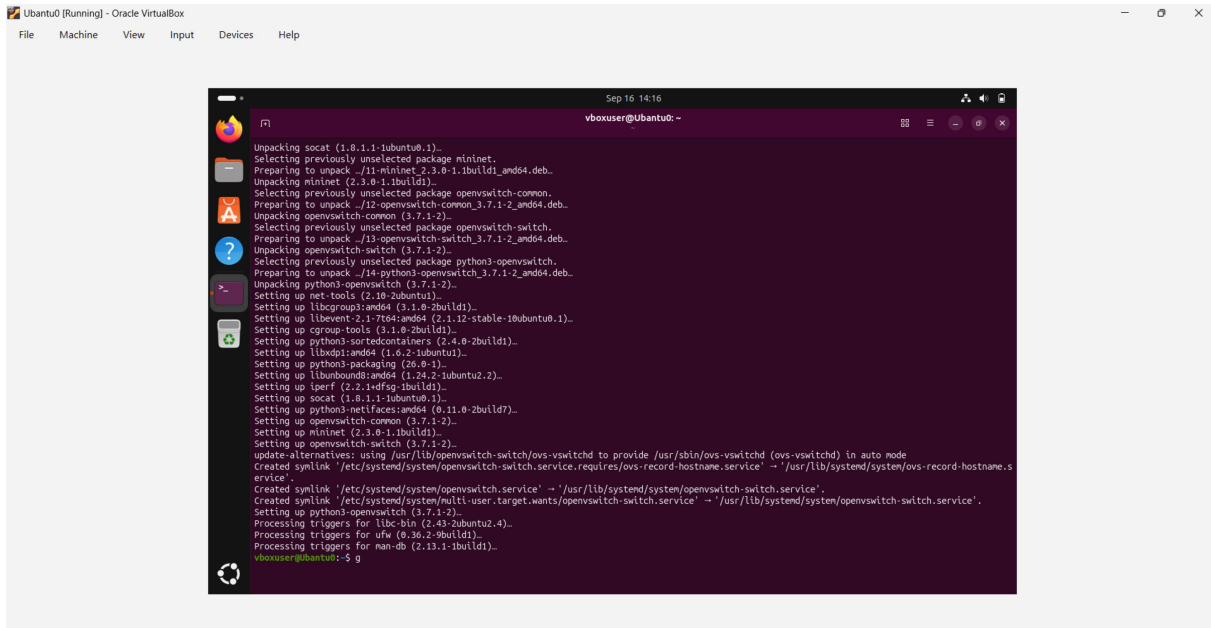
Objective

To create and configure a simple network topology using an open-source network virtualization tool and verify communication between the virtual network nodes.

Task

- Ubuntu Linux Virtual Machine
- VirtualBox
- Mininet
- Create simple Network Topology
- ping and basic Linux networking commands
- After completion, stop the network.

Mininet Installation

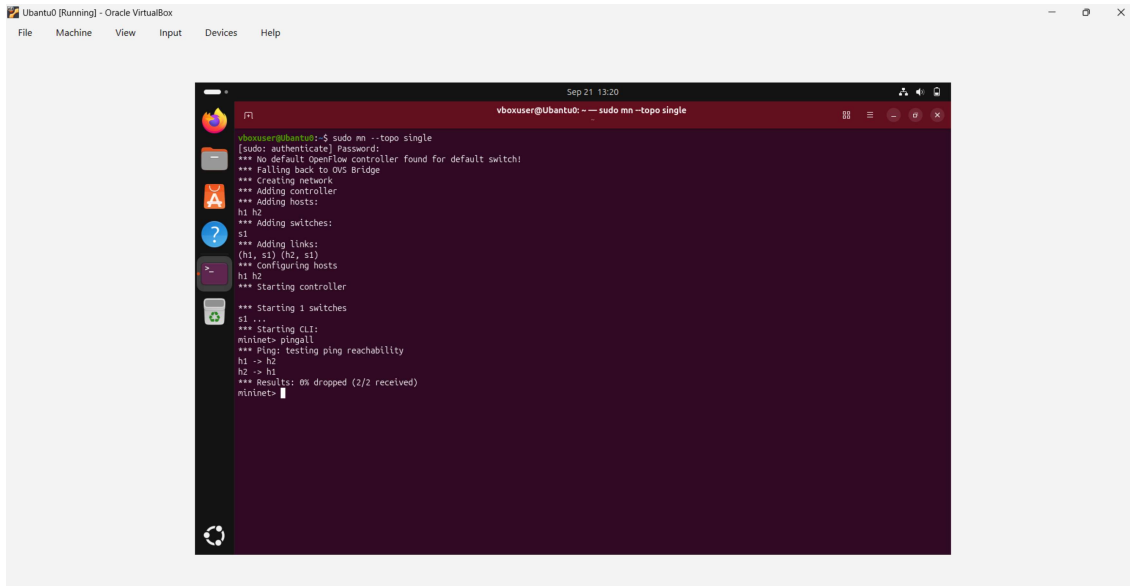


The screenshot shows a terminal window titled 'vboxuser@Ubuntu: ~' with a date and time of 'Sep 16 14:16'. The terminal output displays the installation process for Mininet and Open vSwitch. It starts with unpacking 'socat (1.8.1.1-1ubuntu0.1)', followed by selecting and unpacking 'mininet (2.3.0-1.1build1)'. Then, it selects and unpacks 'openvswitch-common (3.7.1-2)', 'openvswitch-switch (3.7.1-2)', and 'python3-openvswitch (3.7.1-2)'. The process continues with setting up various dependencies like 'libcgroup3:amd64', 'libevent-2.1-7664:amd64', 'cgroup-tools', 'python3-sortedcontainers', 'libxdpi:amd64', 'python3-packaging', 'libunbound:amd64', 'iperf', and 'python3-netifaces:amd64'. It also shows the creation of symlinks for system services and the processing of triggers for 'ufw' and 'man-db'. The terminal ends with the prompt 'vboxuser@ubuntu:~\$ g'.

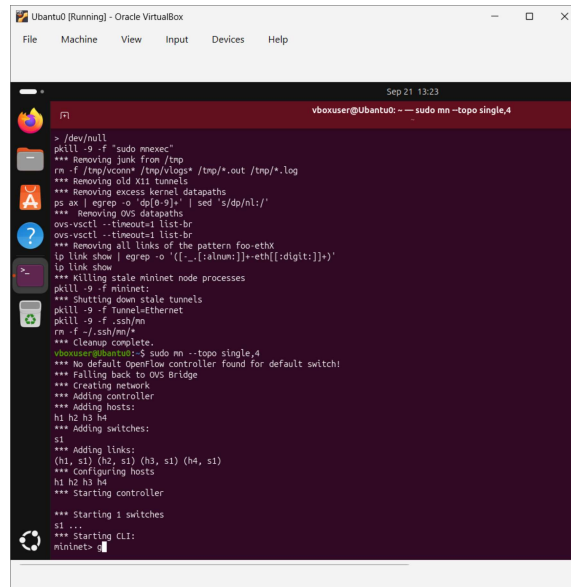
```
Unpacking socat (1.8.1.1-1ubuntu0.1)...
Selecting previously unselected package mininet.
Preparing to unpack ./ii-mininet_2.3.0-1.1build1_amd64.deb...
Unpacking mininet (2.3.0-1.1build1)...
Selecting previously unselected package openvswitch-common.
Preparing to unpack ./i2-openvswitch-common_3.7.1-2_amd64.deb...
Unpacking openvswitch-common (3.7.1-2)...
Selecting previously unselected package openvswitch-switch.
Preparing to unpack ./i3-openvswitch-switch_3.7.1-2_amd64.deb...
Unpacking openvswitch-switch (3.7.1-2)...
Selecting previously unselected package python3-openvswitch.
Preparing to unpack ./i4-python3-openvswitch_3.7.1-2_amd64.deb...
Unpacking python3-openvswitch (3.7.1-2)...
Setting up net-tools (2.10-2ubuntu1)...
Setting up libcgroup3:amd64 (3.1.0-2build1)...
Setting up libevent-2.1-7664:amd64 (2.1.12-stable-10ubuntu0.1)...
Setting up cgroup-tools (3.1.0-2build1)...
Setting up python3-sortedcontainers (2.4.0-2build1)...
Setting up libxdpi:amd64 (1.0.2-1ubuntu1)...
Setting up python3-packaging (20.9-2)...
Setting up libunbound:amd64 (1.24.2-1ubuntu2.2)...
Setting up iperf (2.2.1+dfsg-1build1)...
Setting up socat (1.8.1.1-1ubuntu0.1)...
Setting up python3-netifaces:amd64 (0.11.0-2build7)...
Setting up openvswitch-switch (3.7.1-2)...
Setting up mininet (2.3.0-1.1build1)...
update-alternatives: using /usr/lib/openvswitch-switch/ovs-vsitchd to provide /usr/sbin/ovs-vsitchd (ovs-vsitchd) in auto mode
Created symlink /etc/systemd/system/openvswitch-switch.service.requires/ovs-record-hostname.service' -- /usr/lib/systemd/system/ovs-record-hostname.s
ervice'.
Created symlink /etc/systemd/system/openvswitch.service' -- /usr/lib/systemd/system/openvswitch-switch.service'.
Setting up python3-openvswitch (3.7.1-2)...
Processing triggers for libc-bin (2.43-2ubuntu2.4)...
Processing triggers for ufw (0.36.2-9build1)...
Processing triggers for man-db (2.13.1-1build1)...
vboxuser@ubuntu:~$ g
```

Ubuntu terminal – Mininet/Open vSwitch installation.

1. Star Topology

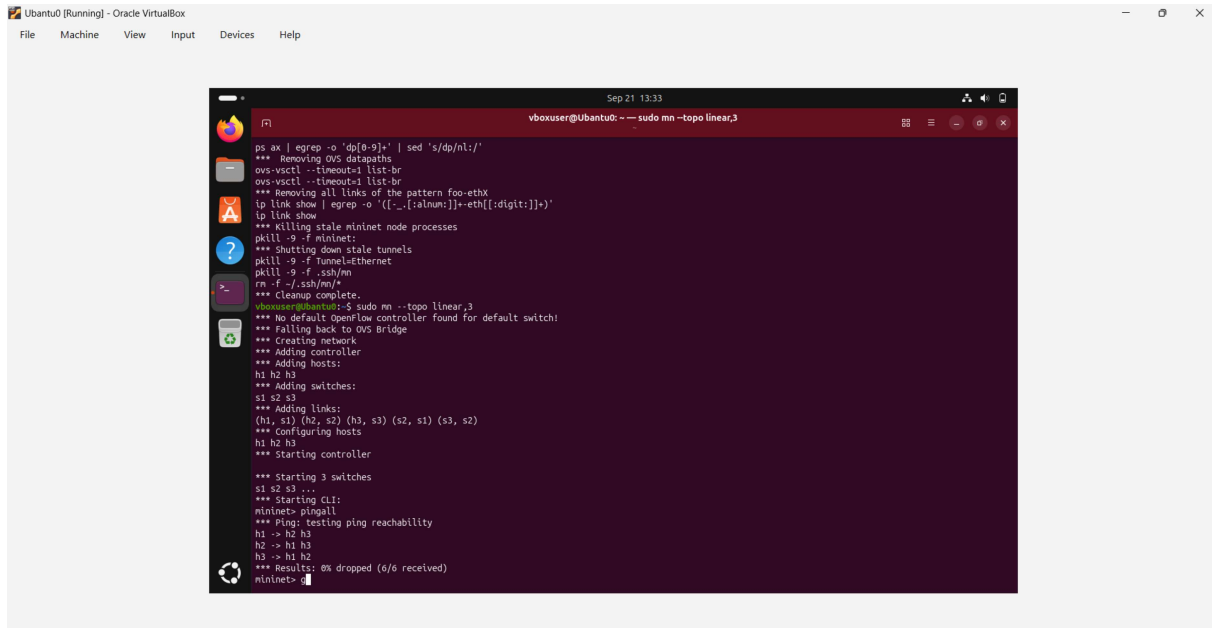


Star with 2 hosts – topology creation and pingall.



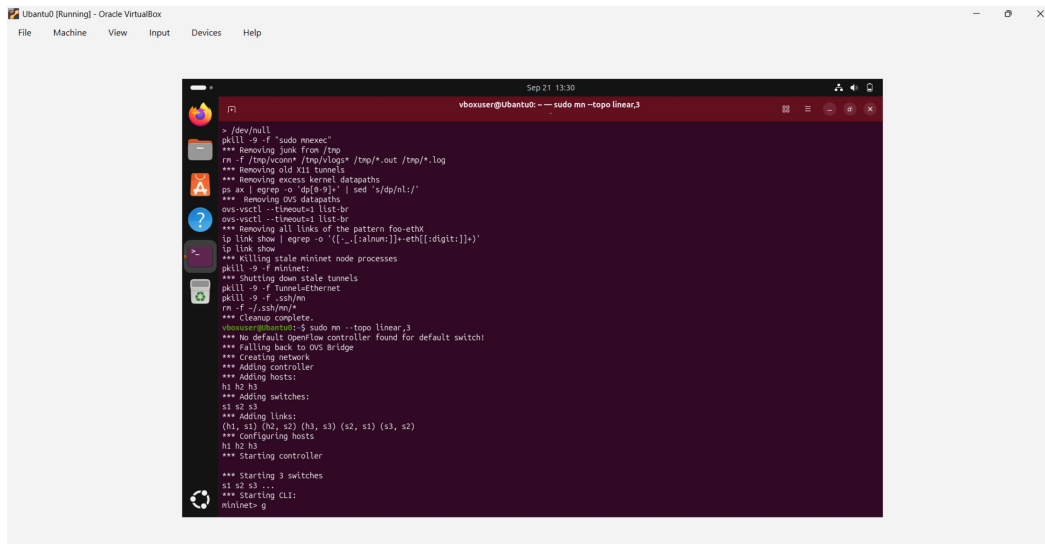
Star with 4 hosts – topology creation.

2. Linear Topology



```
ps ax | egrep -o 'dp[0-9]*' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([_].[a-znum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
killall -9 -f mininet:
*** Shutting down stale tunnels
killall -9 -f Tunnel-ethernet
killall -9 -f .ssh/mn
rm -f -f .ssh/mn/*
*** Cleanup complete.
vboxuser@Ubuntu0:~$ sudo mn --topo linear,3
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1 s2 s3
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (s2, s1) (s3, s2)
*** Configuring hosts
h1 h2 h3
*** Starting controller
*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet> g
```

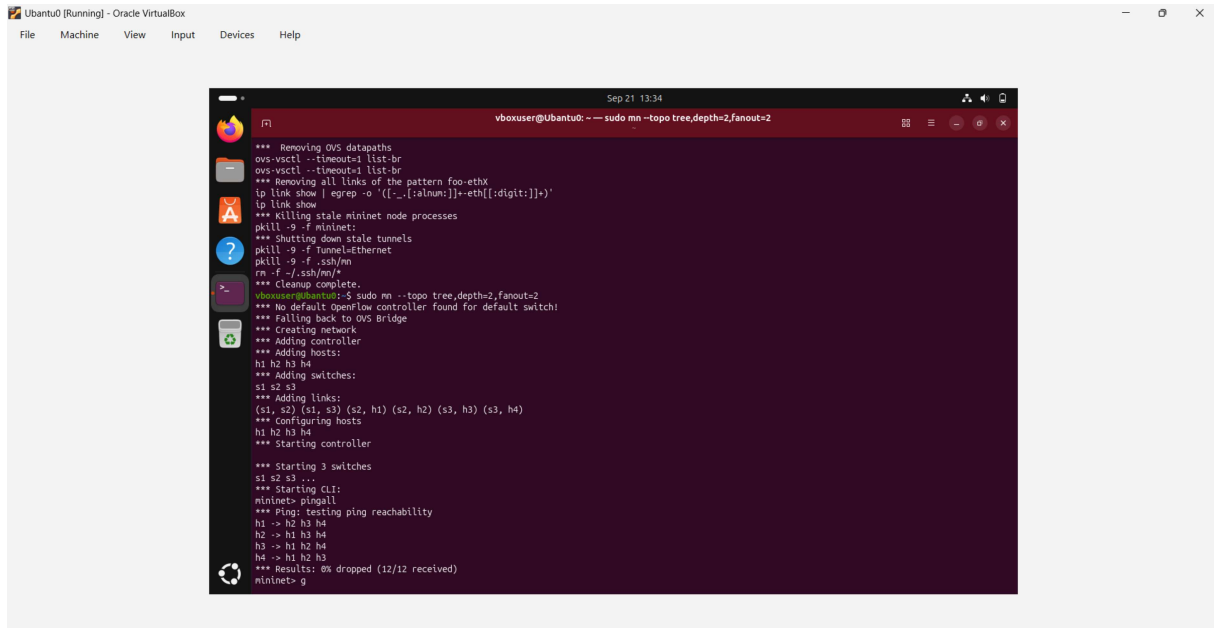
Linear with 3 hosts – successful pingall.



```
> /dev/null
killall -9 -f 'sudo mininet'
*** Removing junk from /tmp
rm -f /tmp/vcon* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old x11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]*' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([_].[a-znum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
killall -9 -f mininet:
*** Shutting down stale tunnels
killall -9 -f Tunnel-ethernet
killall -9 -f .ssh/mn
rm -f -f .ssh/mn/*
*** Cleanup complete.
vboxuser@Ubuntu0:~$ sudo mn --topo linear,3
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1 s2 s3
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (s2, s1) (s3, s2)
*** Configuring hosts
h1 h2 h3
*** Starting controller
*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> g
```

Linear topology configuration.

3. Tree Topology



```

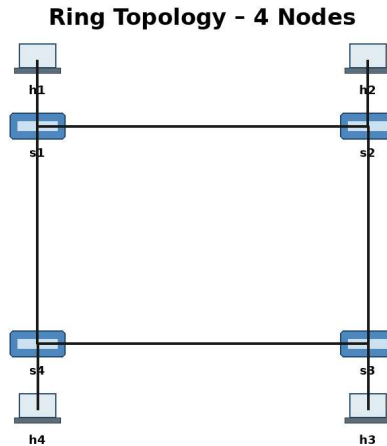
vboxuser@Ubuntu0: ~ -- sudo mn --topo tree,depth=2,fanout=2
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethx
ip link show | grep -o '([[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet
*** Shutting down stale tunnels
pkill -9 -f Tunnel-ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
vboxuser@Ubuntu0: ~$ sudo mn --topo tree,depth=2,fanout=2
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3
*** Adding links:
(s1, s2) (s1, s3) (s2, h1) (s2, h2) (s3, h3) (s3, h4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> g

```

Tree with 4 hosts – successful pingall.

Tree topology diagram is included in the supplied assignment material.

4. Ring Topology



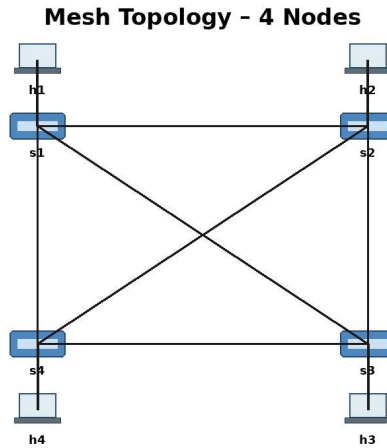
Generated ring topology diagram.

```
Ubuntu0 [Running] - Oracle VirtualBox
vboxuser@Ubuntu0:~$ sudo python3 ring.py

*** Ring topology started successfully.
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
s1-eth2<->s2-eth2 (OK OK)
s2-eth3<->s3-eth2 (OK OK)
s3-eth3<->s4-eth2 (OK OK)
s4-eth3<->s1-eth3 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
```

Generated Ubuntu0 terminal evidence for the ring topology.

5. Mesh Topology



Generated mesh topology diagram.

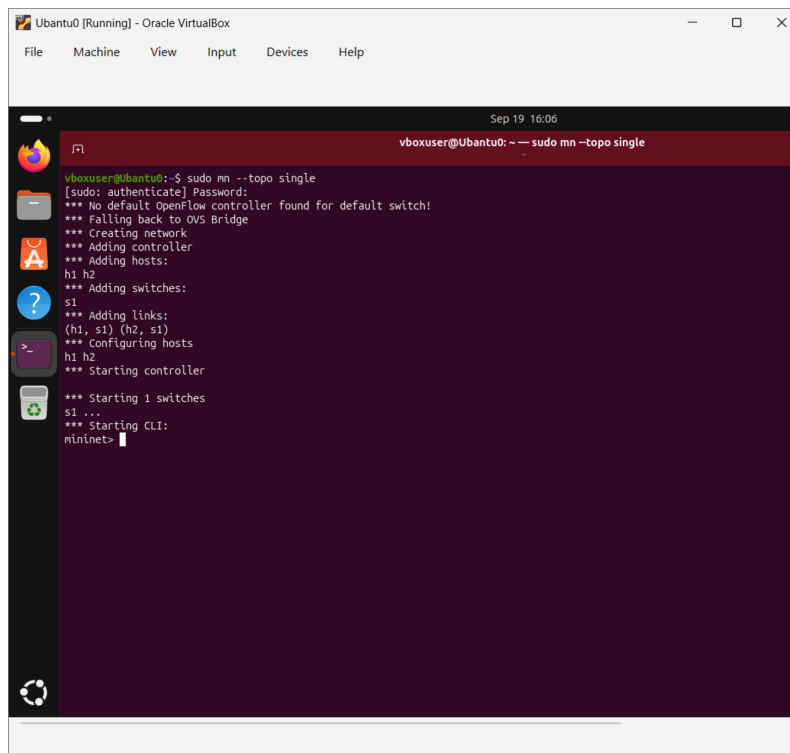
```
Ubuntu0 [Running] - Oracle VirtualBox
vboxuser@Ubuntu0:~$ sudo python3 mesh.py

*** 4-node Mesh topology started successfully.
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
s1-eth2<->s2-eth2 (OK OK)
s1-eth3<->s3-eth2 (OK OK)
s1-eth4<->s4-eth2 (OK OK)
s2-eth3<->s3-eth3 (OK OK)
s2-eth4<->s4-eth3 (OK OK)
s3-eth4<->s4-eth4 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
```

Generated Ubuntu0 terminal evidence for the mesh topology.

6. Verification and Network Stop

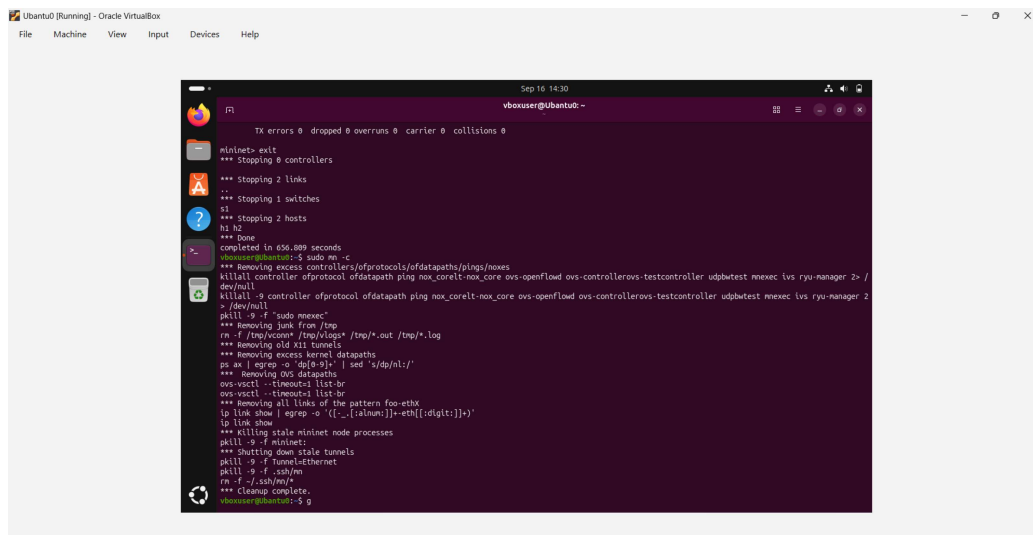
links – view topology links **pingall** – test host reachability **exit** – leave Mininet CLI



The screenshot shows a terminal window titled 'vboxuser@Ubuntu0: ~ -- sudo mn --topo single'. The user has entered the command 'sudo mn --topo single'. The output shows the Mininet CLI starting up, creating a network with two hosts (h1, h2) and one switch (s1). The CLI is now ready for commands.

```
vboxuser@Ubuntu0:~$ sudo mn --topo single
[sudo: authenticate] Password:
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
s1 ...
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Mininet topology execution / CLI evidence.



The screenshot shows a terminal window titled 'vboxuser@Ubuntu0: ~'. The user has entered the command 'mininet exit'. The output shows the Mininet CLI shutting down, removing excess controllers, links, switches, and hosts. The CLI is now ready for commands.

```
mininet> exit
*** Stopping 0 controllers
*** Stopping 2 links
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 656.889 seconds
vboxuser@Ubuntu0:~$ sudo mn -c
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/names
killall controller ofprotocol ofdatapath ping max_corelt-max_core ovs-openflow ovs-controllerovs-testcontroller udpbwtest mnexec lvs ryu-manager 2- /dev/null
killall -9 controller ofprotocol ofdatapath ping max_corelt-max_core ovs-openflow ovs-controllerovs-testcontroller udpbwtest mnexec lvs ryu-manager 2- /dev/null
*** Removing junk from /tmp
rm -f /tmp/acon* /tmp/vlog* /tmp/*out /tmp/*log
*** Removing old xli tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'sd[a-g]*' | sed 's/$(p/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-eth
ip link show | egrep -o '([a-z0-9]*)-eth[0-9]*' | sed 's/$(p/nl:/'
ip link show
*** Killing stale mininet node processes
killall -9 -f mininet
*** Shutting down stale tunnels
killall -9 -f tunneltethernet
killall -9 -f .ssh/mn
rm -f .ssh/mn/*
*** Cleanup complete.
vboxuser@Ubuntu0:~$ g
```

Ubuntu terminal cleanup after the experiment.